Problem Statement:

Create and Art with Neural style transfer on given image using deep learning.

In [1]:
```
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt
from tensorflow.keras.applications import VGG19
from tensorflow.keras.applications.vgg19 import preprocess_input
from tensorflow.keras.preprocessing.image import load_img, img_to_array
```

In [2]:
```
# Load images
def load_image(path):
    img = load_img(path, target_size=(224, 224))
    img = img_to_array(img)
    img = np.expand_dims(img, axis=0)
    return preprocess_input(img)
```

In [3]:
```
content = load_image("contentdc.jpg")
style = load_image("style.jpg")
```

In [4]:
```
# Load VGG model
model = VGG19(weights='imagenet', include_top=False)
```

In [5]:
```
# Select layers
content_layer = 'block5_conv2'
style_layers = ['block1_conv1', 'block2_conv1', 'block3_conv1']
```

In [6]:
```
# Create model
outputs = [model.get_layer(content_layer).output] + \
          [model.get_layer(l).output for l in style_layers]
```

In [7]:
```
model = tf.keras.Model(inputs=model.input, outputs=outputs)
```

In [8]:
```
# Gram matrix
def gram_matrix(x):
```

```
    x = tf.reshape(x, (-1, x.shape[-1]))
    return tf.matmul(x, x, transpose_a=True)
```

In [9]:
```
# Loss functions
def compute_loss(gen):
    outputs = model(gen)
    content_output = outputs[0]
    style_outputs = outputs[1:]

    content_loss = tf.reduce_mean((content_output - model(content)[0])**2)

    style_loss = 0
    for i, style_layer in enumerate(style_outputs):
        style_loss += tf.reduce_mean(
            (gram_matrix(style_layer) - gram_matrix(model(style)[i+1]))**2
        )

    return 10 * content_loss + 500 * style_loss
```

In [10]:
```
# Optimization
generated = tf.Variable(content, dtype=tf.float32)
optimizer = tf.optimizers.Adam(learning_rate=2.0)
```

In [11]:
```
for i in range(100):
    with tf.GradientTape() as tape:
        loss = compute_loss(generated)
    grads = tape.gradient(loss, generated)
    optimizer.apply_gradients([(grads, generated)])
```

In [12]:
```
img = generated.numpy()[0]

# Reverse VGG preprocessing
img[:, :, 0] += 103.939
img[:, :, 1] += 116.779
img[:, :, 2] += 123.68

# Convert BGR → RGB
img = img[:, :, ::-1]

# Clip values
img = np.clip(img, 0, 255).astype('uint8')
```

```
# Display
plt.imshow(img)
plt.axis('off')
```

Out[12]: (np.float64(-0.5), np.float64(223.5), np.float64(223.5), np.float64(-0.5))